



Resource-Constrained Time-Cost Tradeoff Problem in Project Crashing

Presentasi 2

K13 Pemodelan Matematika

MA3251 Pemodelan Matematika
Institut Teknologi Bandung

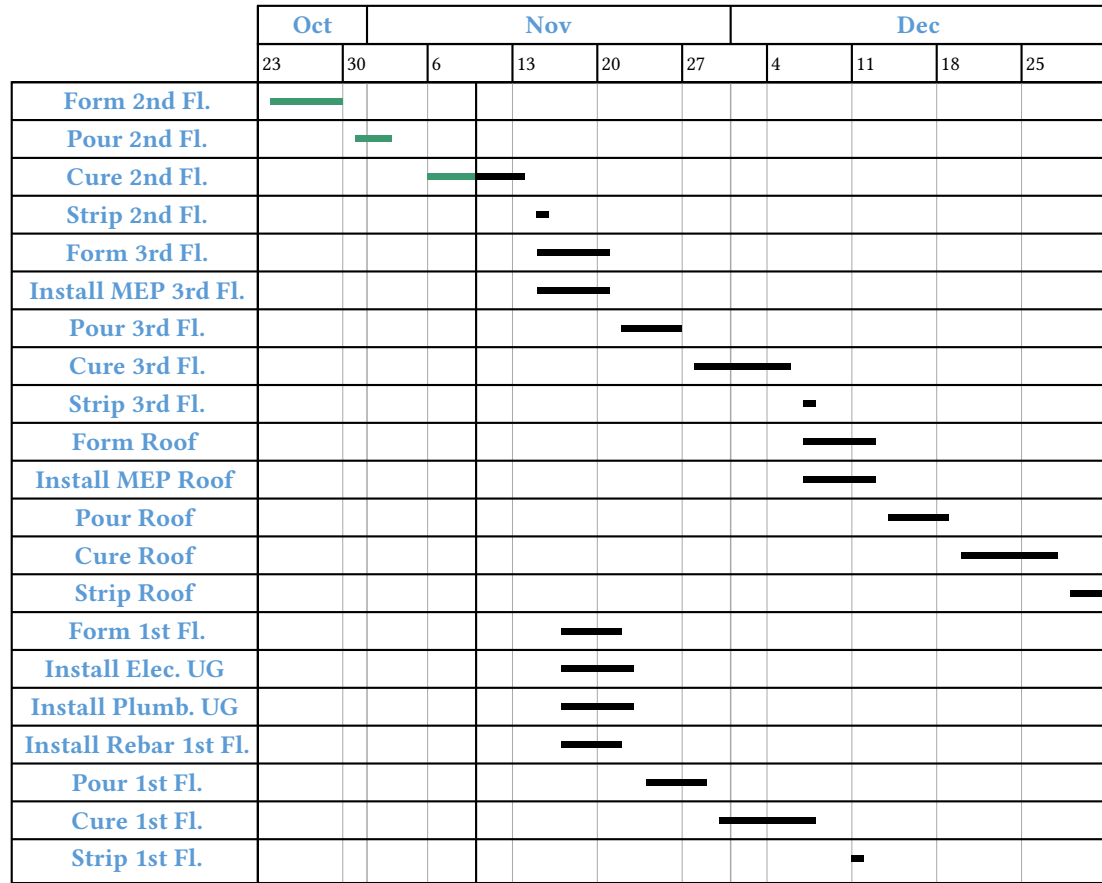
Selasa, 2 Juni 2026

Outline

- Latar Belakang & Masalah
- Skenario 1: Model Baseline (Diskret Linear)
- Skenario 2: Model Baru (Cobb-Douglas)
- Metode Penyelesaian
- Hasil Eksplorasi
- Skenario 3: Model Hybrid
- Kesimpulan
- Rencana Selanjutnya

Latar Belakang & Masalah

Concrete Works Schedule



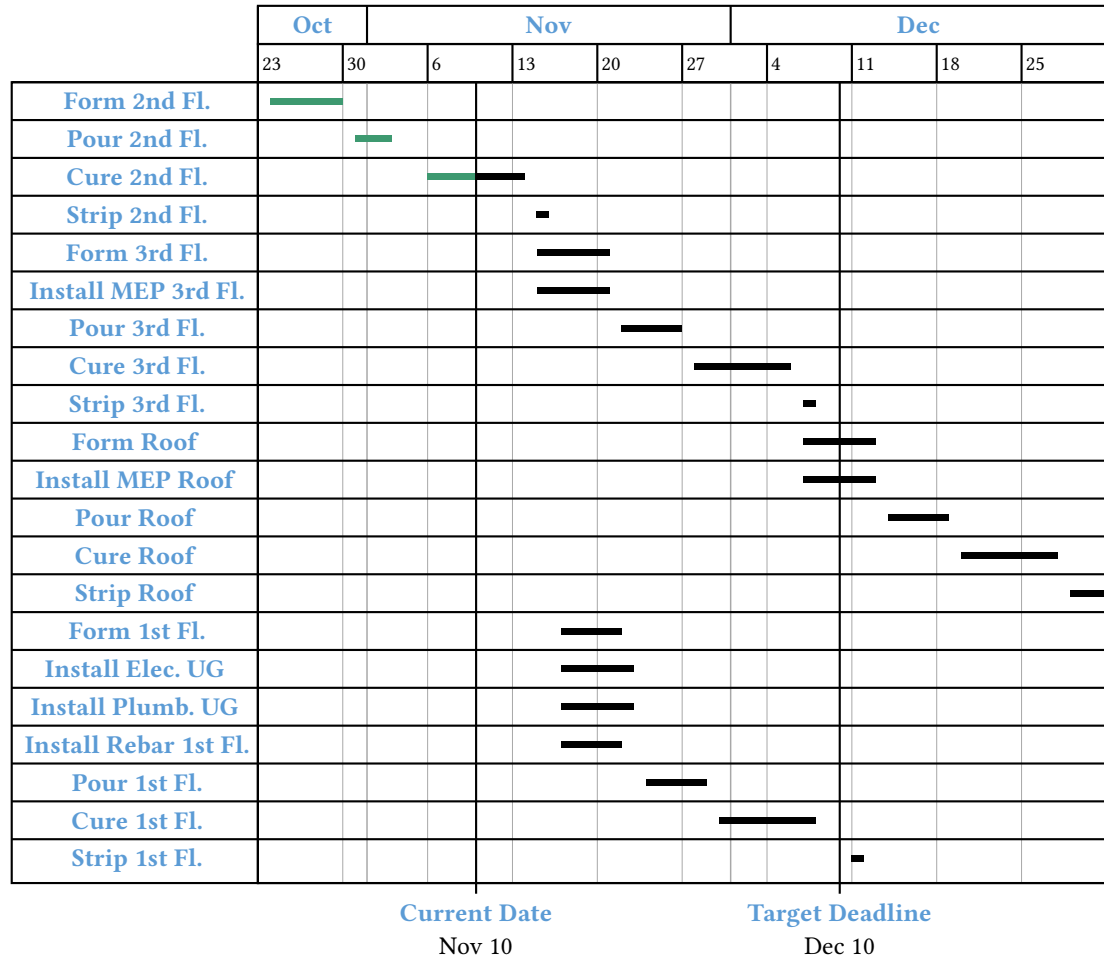
Current Date

Nov 10

Misalkan PT XYZ saat ini sedang melaksanakan proyek pembangunan Gedung ABC.

Kami diberikan **jadwal** terkini proyek mereka. Per 10 November, proyek tersebut diproyeksikan akan selesai pada Desember 30

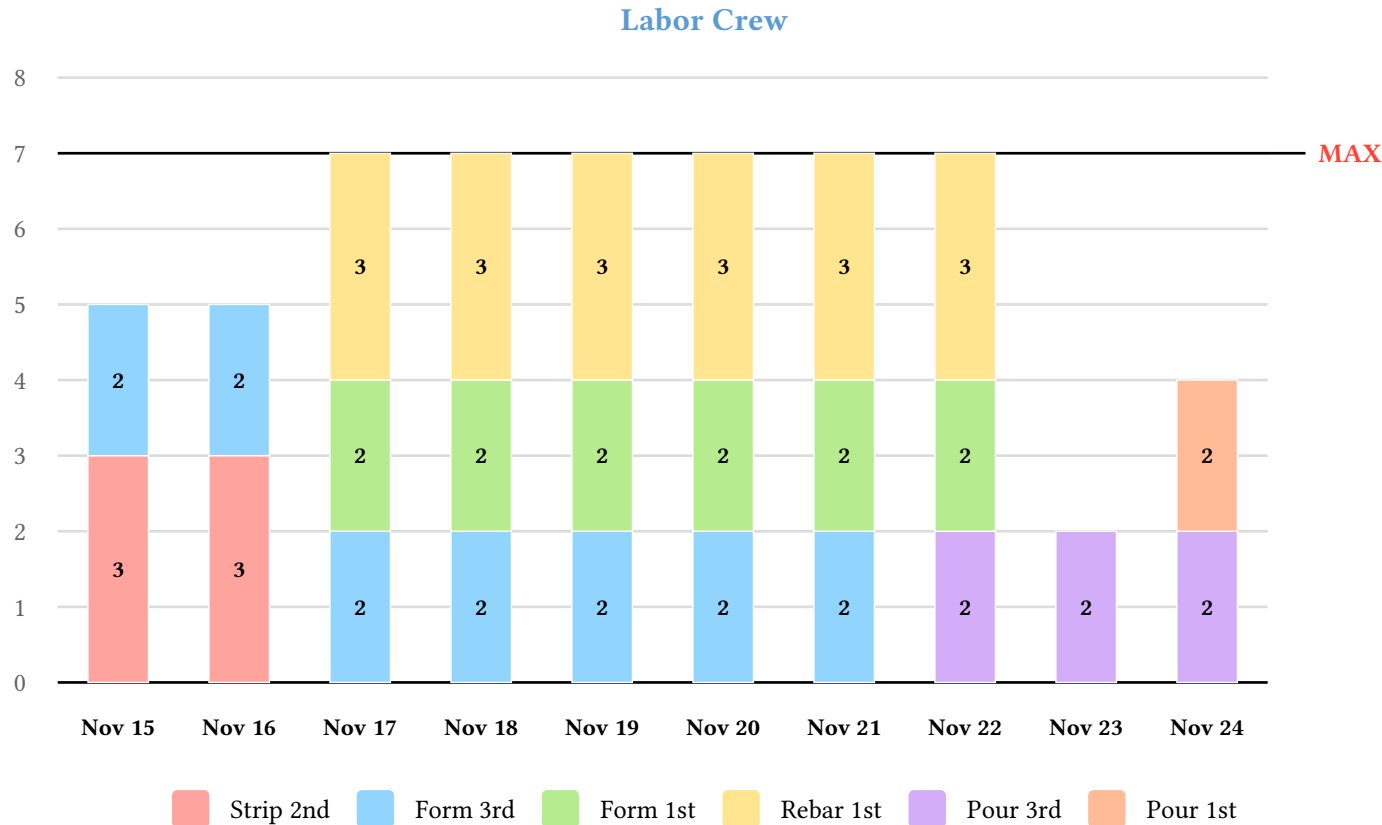
Concrete Works Schedule



Namun, PT XYZ telah menetapkan batas waktu penyelesaian (*deadline*) pada Desember 10.

Oleh karena itu, PT XYZ perlu mempercepat proses pengerjaan proyek tersebut, yang disebut proses *crashing*.

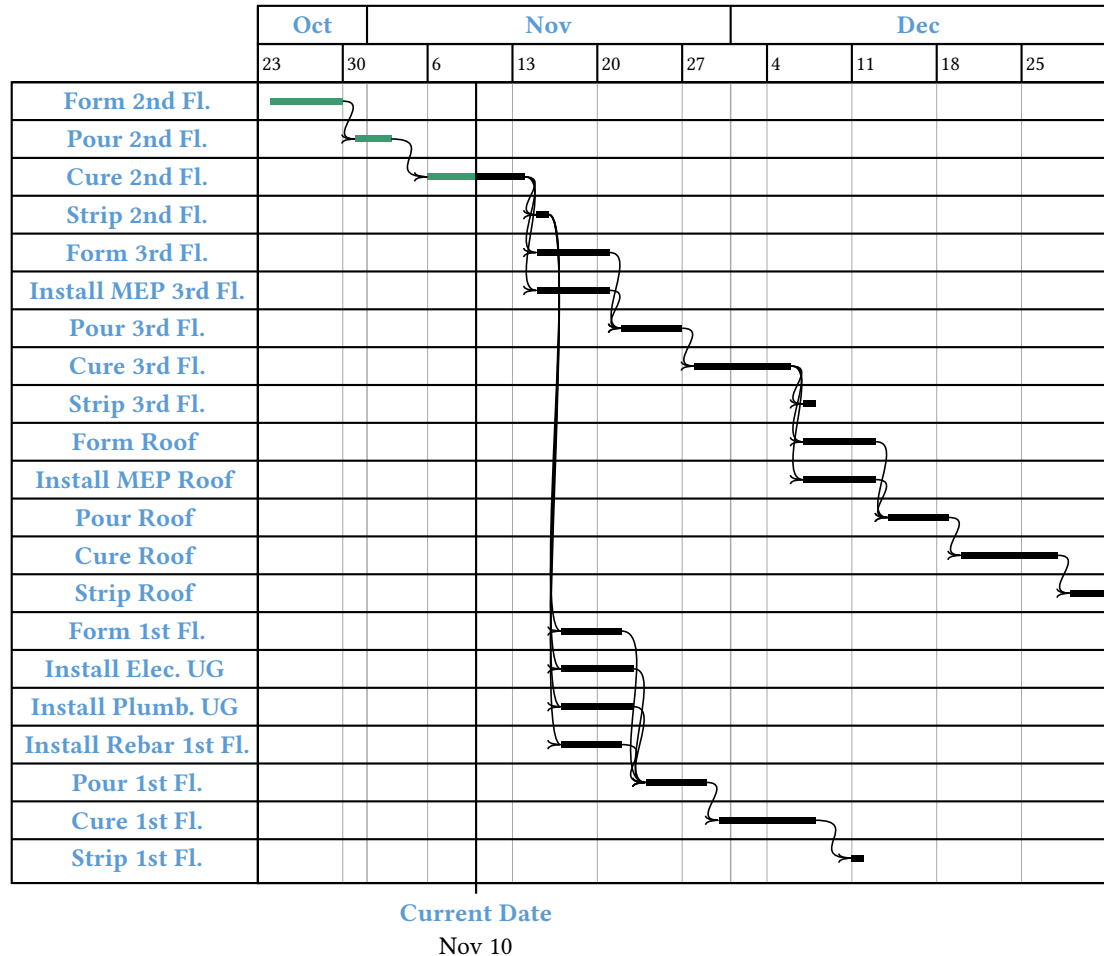
Terkait hal ini, PT XYZ meminta kami untuk menentukan strategi crashing yang paling optimal.



Proyek pembangunan memiliki berbagai batasan, salah satunya adalah batasan **sumber daya**.

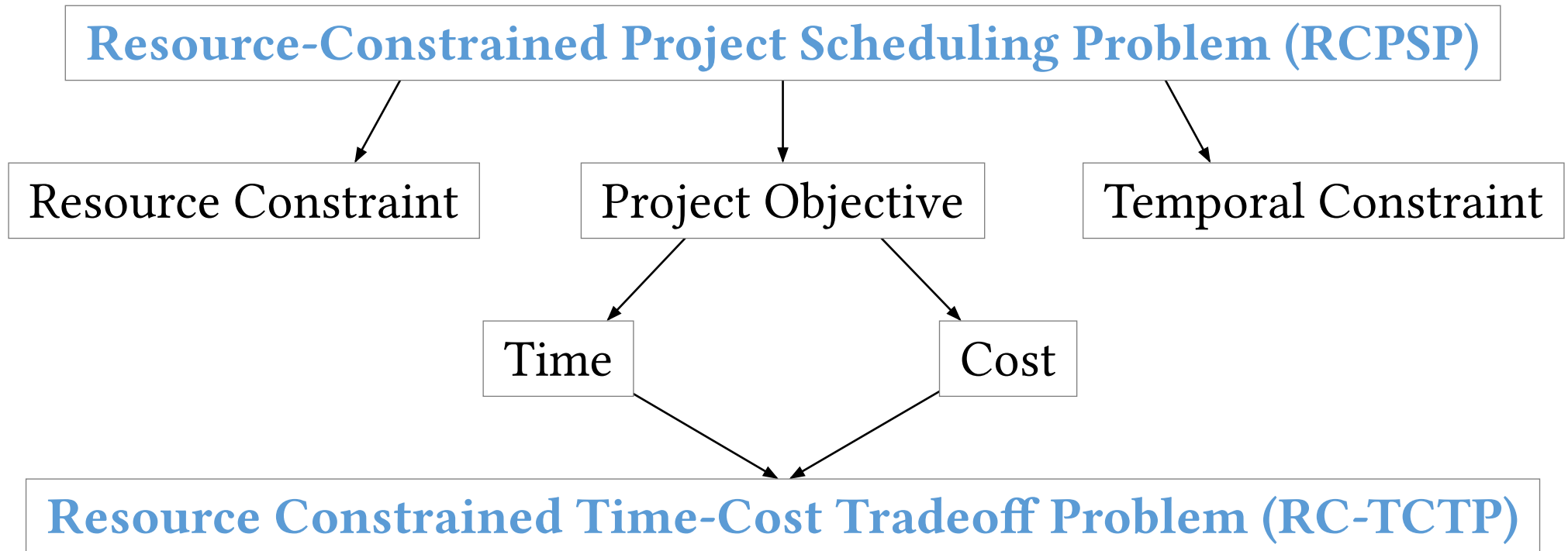
Dalam kasus ini, sebagai contoh, alokasi labor crew di lapangan per hari untuk setiap task dengan batasan maksimal 7 pekerja per hari, dapat dilihat pada grafik.

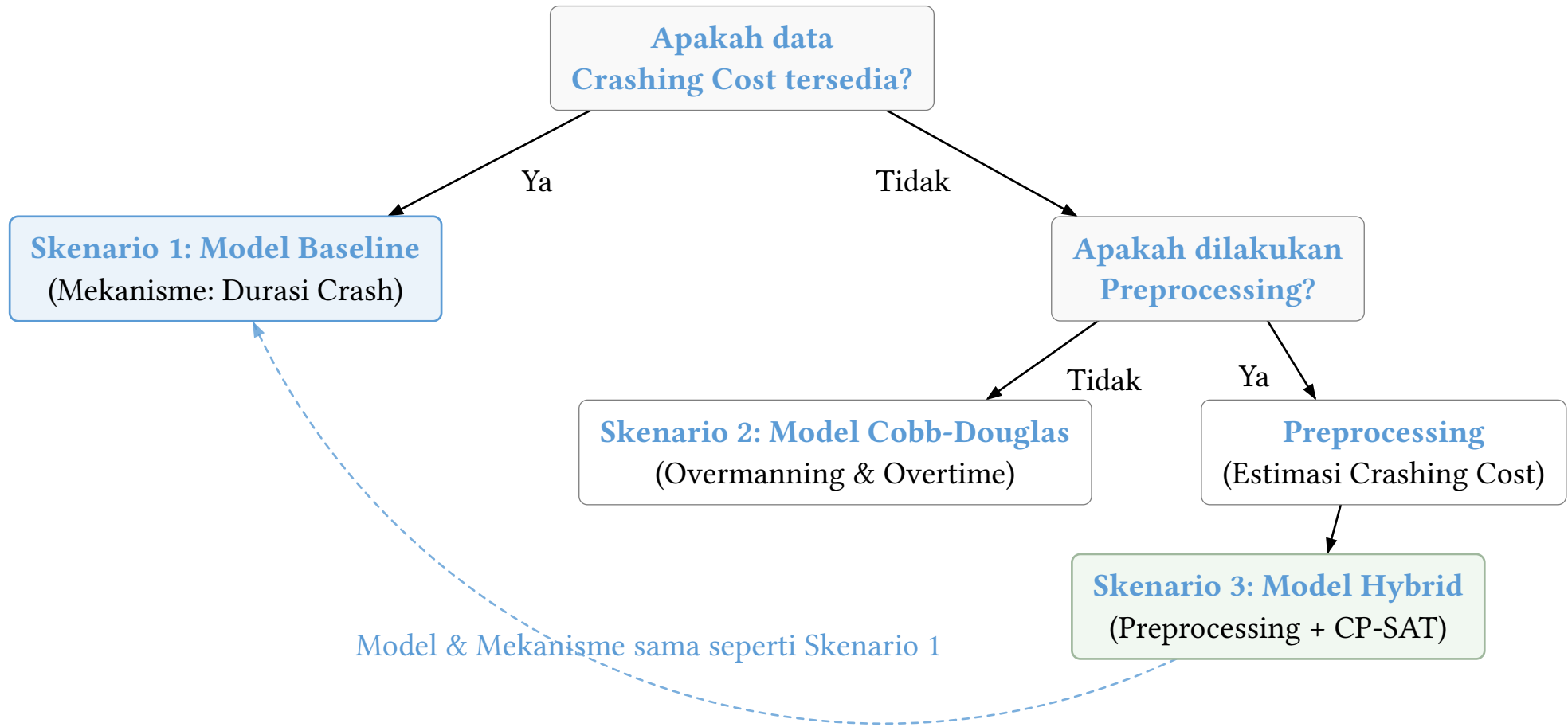
Temporal Constraints



Selain itu, ada juga batasan **waktu**. Batasan ini berupa ketergantungan (*dependencies*) antara setiap task pada proyek.

Jadi, suatu task tidak bisa mulai atau berakhir sebelum beberapa task-task tertentu belum selesai atau memulai. Secara khusus, ini sering disebut *precedence constraint*.





Contoh Data yang Dimiliki (1)

Dalam Skenario 1 (Baseline), kita memiliki data:

- **Data Aktivitas:** Durasi normal ($d_i^{\max}$), durasi minimum ($d_i^{\min}$), dan biaya crashing harian (C_i).
- **Data Sumber Daya:** Kapasitas harian ($U_k^{\max}$) dan kebutuhan harian ($U_{i,k}$).

Nama Aktivitas	Precedence	$d_i^{\min}$	$d_i^{\max}$	Normal Cost	C_i (USD/hari)	Sumber Daya
Bids & Contracts	-	7 hari	10 hari	\$600	\$60.00	Gen. Mgmt (1), Proj. Mgmt (1)
Grading & Permits	Bids	7 hari	10 hari	\$700	\$70.00	Gen. Mgmt (1), Survey Crew (1)
Site Work	Grading	5 hari	7 hari	\$300	\$30.00	Labor Crew (3), Contractor (2)
...						

Resource Name	Availability
General Manager	1 orang
Project Manager	1 orang
Survey Crew	1 orang
Labor Crew	5 orang
Contractor	3 orang
...	

Contoh: *Site Work* dikerjakan setelah *Grading & Permits* oleh **3 Labor Crew** dan **2 Contractor** selama **7** hari dengan biaya \$300. Namun dapat dipercepat hingga **5** hari dengan biaya percepatan \$30 per hari.

Total terdapat **5 Labor Crew** yang dan **3 Contractor** yang tersedia dan dapat dikerahkan untuk menyelesaikan Task tersebut.

Contoh Data yang Dimiliki (2)

Dalam Skenario 2 & 3, terdapat

- **task_table**: ID, Nama, Durasi Baseline, Outline Level, Predecessors.
- **resource_table**: Tarif reguler (r_k) dan kapasitas maksimal harian ($U_k^{\max}$).
- **assignment_table**: Total usaha kerja ($W_{i,k}$ dalam jam) dan persentase alokasi harian baseline ($U_{i,k}$).

ID	Aktivitas	Baseline	Pred.	Resource (SDM)	Jam Kerja
2	Receive notice to proceed and sign contract	3 hari	-	G.C. General Management	24 jam
3	Submit bond and insurance documents	2 hari	2	G.C. Project Management	16 jam
				G.C. General Management	4 jam
4	Prepare and submit project schedule	2 hari	3	G.C. Project Management	4 jam
				G.C. Scheduler	16 jam
...					

ID	SDM (Resource)	Tarif
1	G.C. General Management	\$120.00/jam
2	G.C. Project Management	\$95.00/jam
9	G.C. Labor Crew	\$30.00/jam
...		

Contoh: Task *Submit bond and insurance documents* normalnya membutuhkan **2** hari, dikerjakan oleh *Project Manager* **8** jam kerja per hari ($8 \times 2 = 16$ total jam kerja) dan *General Manager* **2** jam per hari ($2 \times 2 = 4$ total jam kerja). Task ini harus dikerjakan setelah task dengan ID=2 yaitu *Recieve notice to proceed and sign contract*.

Tarif *General Manager* adalah \$120 per jam kerja dan *Project Manager* adalah \$95 per jam kerja.

Skenario 1: Model Baseline (Diskret Linear)

Variabel & Himpunan

Notasi	Deskripsi	Satuan
I	Himpunan aktivitas proyek	-
K	Himpunan jenis sumber daya	-
E	Himpunan relasi precedence	-
I_0, I_1	Aktivitas selesai / belum mulai	-
I_0^C	Aktivitas belum selesai	-
s_i	Hari mulai aktivitas i	hari
e_i	Hari selesai aktivitas i	hari
s_{n+1}	Hari penyelesaian proyek	hari

Parameter Model

Notasi	Deskripsi	Satuan
$d_i^{\max}$	Durasi normal aktivitas i	hari
$d_i^{\min}$	Durasi minimum aktivitas i	hari
C_i	Biaya percepatan aktivitas i	\$/hari
$U_{i,k}$	Kebutuhan resource k oleh i	pekerja
$U_k^{\max}$	Kapasitas maksimum resource k	pekerja
T_0	Hari peninjauan status proyek	hari
$T_{\max}$	Target tenggat waktu proyek	hari
c_{late}	Koefisien denda keterlambatan	\$/hari
c_{early}	Koefisien bonus penyelesaian awal	\$/hari

Model baseline merumuskan masalah optimisasi penjadwalan dengan durasi diskrit dan biaya percepatan durasi *crashing cost* linier menggunakan parameter yang diketahui sejak awal.

Mekanisme Dasar

Crashing dilakukan secara langsung dengan menambah biaya, tanpa memedulikan *underlying mechanism* dari proses *crashing*.

Non-Preemptive

Aktivitas yang sedang berjalan tidak dapat diinterupsi.

Linieritas Biaya Crashing

Pengurangan durasi aktivitas diasumsikan memiliki biaya tambahan konstan per hari.

Ketersediaan Resource

Resource *availability* dari masing-masing resource dianggap konstan.

Variabel Keputusan	Domain	Deskripsi
s_i	$\mathbb{Z}$	Hari mulai untuk aktivitas $i \in I$.
e_i	$\mathbb{Z}$	Hari akhir untuk aktivitas $i \in I$.

Variabel Global Proyek:

$$s_{n+1} \in \mathbb{Z}$$

s_{n+1} adalah **task semu** yang merepresentasikan penyelesaian proyek.

Batasan Precedence: $s_{n+1} \geq e_i$ untuk semua aktivitas i sehingga s_{n+1} akan bernilai sama dengan waktu penyelesaian proyek.

Temporal Constraints

Skenario 1: Model Baseline (Diskret Linear)

Batasan Durasi & Precedence:

Durasi pengerjaan aktivitas dibatasi antara durasi minimum setelah crashing dan durasi normalnya.

$$d_i^{\min} \leq e_i - s_i \leq d_i^{\max} \quad \forall i \in I$$

Aktivitas penerus hanya boleh dimulai setelah seluruh aktivitas pendahulunya selesai dikerjakan.

$$s_i \geq e_j \quad \forall (i, j) \in E$$

Keterangan:

1. $d_i^{\min}$ menyatakan *minimum crashable duration*
2. $d_i^{\max}$ menyatakan *normal duration*
3. T_0 menyatakan *current date of the project*
4. $s_i^{(0)}$ dan $e_i^{(0)}$ menyatakan jadwal baseline tanpa crashing

Batasan Dinamis terhadap Hari Peninjauan (T_0):

- **Selesai ($i \in I_0$):**

Jadwal dikunci sesuai realisasi historisnya.

$$s_i = s_i^{(0)}, \quad e_i = e_i^{(0)}$$

- **Berjalan ($i \in I_0^C \cap I_1^C$):**

Hari mulai dikunci dan penyelesaian dibatasi setelah T_0 .

$$s_i = s_i^{(0)}, \quad e_i \geq T_0$$

- **Belum Mulai ($i \in I_1$):**

Tugas hanya boleh dijadwalkan pada atau setelah T_0 .

$$s_i \geq T_0$$

Jumlah kebutuhan harian untuk setiap jenis sumber daya yang sedang aktif digunakan secara bersamaan tidak boleh melebihi kapasitas maksimum yang tersedia.

$$\sum_{i \in I} U_{i,k} \cdot \mathbb{1}\{s_i \leq s_j < e_i\} \leq U_k^{\max} \quad \forall k \in K, \forall j \in I$$

Penjelasan:

- Himpunan I merupakan indeks aktivitas dan K merupakan indeks resource.
- Parameter $U_{i,k}$ menyatakan kebutuhan harian sumber daya k oleh aktivitas i , dan $U_k^{\max}$ menyatakan kapasitas harian maksimum sumber daya k .
- Fungsi indikator $\mathbb{1}\{s_i \leq s_j < e_i\}$ bernilai 1 jika aktivitas i sedang berjalan pada saat aktivitas j dimulai (s_j), memastikan total penggunaan pekerja di setiap titik awal aktivitas selalu berada dalam batas kapasitas.

A. Multi-Objektif

Optimisasi multiobjektif atas waktu dan biaya:

$$\min \left(s_{n+1}, \sum_{i \in I_0^C} C_i (d_i^{\max} - (e_i - s_i)) \right)$$

1. Cost-Driven (Deadline Terkunci)

Meminimalkan total biaya crash dengan batas deadline:

$$\min \sum_{i \in I_0^C} C_i (d_i^{\max} - (e_i - s_i)) \quad \text{s.t.} \quad s_{n+1} \leq T_{\max}$$

2. Time-Driven (Anggaran Terkunci)

Meminimalkan durasi proyek dengan batas anggaran B :

$$\min s_{n+1} \quad \text{s.t.} \quad \sum_{i \in I} C_i (d_i^{\max} - (e_i - s_i)) \leq B$$

B. Fungsi Objektif Bonus-Penalty

Dimensi “time” dikonversikan menjadi “cost” melalui mekanisme bonus dan penalty pada fungsi objektif *cost-driven*.

$$\min \underbrace{\sum_{i \in I_0^C} C_i (d_i^{\max} - (e_i - s_i))}_{\text{Biaya Crashing Baru}} + \underbrace{c_{\text{late}} \max(0, s_{n+1} - T_{\max})}_{\text{Denda Keterlambatan}} - \underbrace{c_{\text{early}} \max(0, T_{\max} - s_{n+1})}_{\text{Bonus Penyelesaian Awal}}$$

Skenario 2: Model Baru (Cobb-Douglas)

Variabel & Himpunan

Notasi	Deskripsi	Satuan
I, K	Himpunan aktivitas & resource	-
E_{FS}, E_{SS}, E_{FF}	Relasi precedence FS, SS, FF	-
I_0, I_1, I_0^C	Status aktivitas proyek	-
s_i, e_i	Waktu mulai & selesai aktivitas i	hari
$x_{i,k}$	Pengali overmanning pekerja k	-
$\tau_{i,k}$	Jam lembur harian pekerja k	jam/hari
$d_{i,k}$	Durasi aktual pekerja k pada i	hari
$z_{i,k}$	Total biaya pekerja k pada i	\$
s_{n+1}	Hari penyelesaian proyek	hari

Parameter Model

Notasi	Deskripsi	Satuan
$d_{i,k}^{(0)}$	Durasi normal pekerja k pada i	hari
$W_{i,k}^{(0)}$	Usaha kerja normal pekerja k	hari-orang
$p_{i,k}$	Progress pengerjaan pekerja k	%
r_k, r'_k	Tarif upah normal & lembur	\$/jam
α, β	Elastisitas crowding & lembur	-
$x_{i,k}^{\max}$	Batas pengali overmanning	-
δ_{ij}	Waktu lag/lead aktivitas i & j	hari
$T_0, T_{\max}$	Hari peninjauan & target deadline	hari
$c_{\text{late}}, c_{\text{early}}$	Koefisien denda & bonus proyek	\$/hari
$U_{i,k}, U_k^{\max}$	Kebutuhan & kapasitas resource	pekerja

Sekarang, kita tidak diberikan **crashing cost** secara langsung. Apa yang harus dilakukan?

Formulasi Durasi Berdasarkan Data

Secara fisik, untuk jenis tenaga kerja k pada tugas i , total jam kerja (usaha kerja $W_{i,k}$) adalah hasil perkalian antara hari kerja ($d_{i,k}$), **durasi kerja** harian normal (8 jam), dan **jumlah tenaga kerja** baseline ($U_{i,k}$):

$$W_{i,k} = d_{i,k} \cdot 8 \cdot U_{i,k} \quad \rightarrow \quad d_{i,k} = \frac{W_{i,k}}{8 \cdot U_{i,k}}$$

sedangkan biaya *baseline* adalah

$$z_{i,k}^{(0)} = W_{i,k} r_k.$$

Mekanisme Crashing

Karena kedua variabel di atas yang memengaruhi durasi, kita pilih keduanya sebagai mekanisme crashing-nya: kita bisa menambahkan tenaga kerja (**overmanning**, $x_{i,k} \geq 1.0$ dengan batas $x_{\max} = 2.0$) dan/atau menambahkan jam kerja lembur (**overtime**, $\tau_{i,k} \geq 0.0$ dengan batas $\tau_{\max} = 4.0$ jam/hari).

Paradoks Biaya Nol (Naive Crashing)

Secara naif, durasi task dapat dihitung dengan

$$d'_{i,k} = \frac{W_{i,k}}{(8 + \tau_{i,k})x_{i,k}U_{i,k}}$$

Namun,

$$z_{i,k} = d'_{i,k} \cdot x_{i,k}U_{i,k} \cdot (8 + \tau_{i,k})r_k = W_{i,k}r_k = z_{i,k}^{(0)}.$$

Hal ini tidak realistis karena mengabaikan hilangnya efisiensi akibat kepadatan area kerja (**crowding**) dan kelelahan pekerja (**fatigue**).



Deminishing Return via Fungsi Cobb-Douglas

Teori ekonomi menggunakan fungsi produksi Cobb-Douglas

$$Y = AL^\alpha K^\beta$$

untuk memodelkan hubungan output (Y) dengan tenaga kerja (L) dan modal (K). Kita memetakan:

- Input Tenaga Kerja (L) → **overmanning**
- Input Waktu/Modal (K) → **overtime**

Dengan eksponen elastisitas marjinal $\alpha, \beta \in (0, 1)$ untuk memodelkan **diminishing returns**:

$$d'_{i,k} = d_{i,k}^{(0)} \cdot \left(\frac{1}{x_{i,k}} \right)^\alpha \cdot \left(\frac{8}{8 + \tau_{i,k}} \right)^\beta$$

Formulasi Biaya Harian:

Biaya total baru adalah hasil kali dari durasi kerja aktual, jumlah pekerja harian, dan tarif upah harian yang mencakup premi tarif lembur (r'_k):

$$\begin{aligned} z_{i,k}(x_{i,k}, \tau_{i,k}) &= d_{i,k}(x_{i,k}, \tau_{i,k}) \cdot x_{i,k} U_{i,k} \cdot (8r_k + \tau_{i,k} r'_k) \\ &= \frac{W_{i,k}}{8U_{i,k}} \cdot \left(\frac{1}{x_{i,k}} \right)^\alpha \cdot \left(\frac{8}{8 + \tau_{i,k}} \right)^\beta \cdot x_{i,k} U_{i,k} \cdot (8r_k + \tau_{i,k} r'_k) \\ &= W_{i,k} \cdot x_{i,k}^{-\alpha} \cdot x_{i,k} \cdot \left(\frac{8 + \tau_{i,k}}{8} \right)^{-\beta} \cdot \frac{8 + \tau_{i,k}}{8} \cdot \frac{1}{8 + \tau_{i,k}} \cdot (8r_k + \tau_{i,k} r'_k) \\ &= \underbrace{W_{i,k} r_k}_{\text{baseline}} \cdot \underbrace{x_{i,k}^{1-\alpha}}_{\text{overman}} \cdot \underbrace{\left(\frac{8 + \tau_{i,k}}{8} \right)^{1-\beta}}_{\text{overtime}} \cdot \underbrace{\frac{8 + \frac{r'_k}{r_k} \tau_{i,k}}{8 + \tau_{i,k}}}_{\text{extra wage}}. \end{aligned}$$

Berdasarkan penurunan mekanisme kerja yang telah dilakukan sebelumnya, model ini mengasumsikan beberapa hal secara implisit mengenai sifat *project crashing* yang dilakukan.

Mekanisme Dasar

Crashing dilakukan secara tidak langsung melalui dua mekanisme: *overtime* (lembur) dan *overmanning* (penambahan pekerja).

Non-Preemptive

Aktivitas yang sedang berjalan tidak dapat diinterupsi. Tetapi, suatu sumber daya (pekerja) dapat berpindah antar aktivitas.

Non-Linieritas Biaya Crashing

Diasumsikan hubungan dua variabel mekanisme berhubungan tak linear dengan durasi (sehingga juga dengan biaya) berdasarkan aturan Cobb-Douglas.

Ketersediaan Resource

Resource *availability* dari masing-masing resource dianggap konstan. Tetapi, resource *requirement* dari masing-masing task bisa berubah.

Solver menentukan waktu mulai, tingkat overmanning, dan jam lembur untuk menghasilkan jadwal proyek optimal.

Variabel Keputusan	Domain	Deskripsi
s_i	$\mathbb{R}$	Hari mulai untuk aktivitas $i \in I$.
$x_{i,k}$	$\mathbb{R}$	Pengali overmanning pekerja k pada aktivitas i ($1 \leq x_{i,k} \leq x_{i,k}^{\max}$).
$\tau_{i,k}$	$\mathbb{Z}$	Jam lembur harian pekerja k pada aktivitas i ($0 \leq \tau_{i,k} \leq 4$).

Variabel Global Proyek:

$$s_{n+1} \in \mathbb{R}$$

s_{n+1} adalah **task semu** yang merepresentasikan penyelesaian proyek.

Batasan Precedence: $s_{n+1} \geq e_i$ untuk semua aktivitas i sehingga s_{n+1} akan bernilai sama dengan waktu penyelesaian proyek.

Temporal Constraints

Batasan Durasi

Di sini, e_i hanya variabel pembantu (bukan variabel keputusan) untuk memudahkan penulisan batasan.

$$e_i = s_i + \max_k d'_{i,k}$$

Batasan Precedence

Seperti sebelumnya, perlu ada batasan *precedence*.

$$s_j \geq e_i \quad \forall (i, j) \in E$$

Keterangan:

1. p_i menyatakan proporsi task i yang sudah selesai
2. T_0 menyatakan *current date of the project*
3. $s_i^{(0)}$ menyatakan jadwal baseline tanpa crashing
4. $W_{i,k}^{(0)}$ menyatakan total jam kerja baseline tanpa crashing

Skenario 2: Model Baru (Cobb-Douglas)

Batasan Dinamis

- **Selesai ($i \in I_0$):** Jadwal dan alokasi tugas **dikunci** sepenuhnya.

$$s_i = s_i^{(0)},$$

$$x_{i,k} = 1,$$

$$\tau_{i,k} = 0$$

- **Berjalan ($i \in I_0^C \cap I_1^C$):** Sisa alokasi dioptimasi dari sisa usaha kerja.

$$s_i = s_i^{(0)},$$

$$e_i \geq T_0,$$

$$W_{i,k} = W_{i,k}^{(0)}(1 - p_i)$$

- **Belum Mulai ($i \in I_1$):** Tugas dimulai pada atau setelah T_0 .

$$s_i \geq T_0,$$

$$W_{i,k} = W_{i,k}^{(0)}$$

Total alokasi pekerja harian untuk suatu jenis sumber daya tidak boleh melebihi kapasitas maksimum yang tersedia sepanjang durasi tugas mereka.

$$\sum_{i \in I} U_{i,k} \cdot \mathbb{1}\{s_i \leq s_j < s_i + d_{i,k}\} \leq U_k^{\max} \quad \forall k \in K, \forall j \in I$$

Penjelasan:

- Batasan ini dievaluasi pada level alokasi pekerja ($s_i + d_{i,k}$) bukan tingkat tugas (e_i), yang memungkinkan pekerja berpindah tugas begitu pekerjaan spesifiknya selesai.
- Pengecekan hanya dilakukan pada setiap titik hari mulai aktivitas (s_j) untuk menyederhanakan perhitungan beban komputasi.

A. Multi-Objektif

Optimisasi multiobjektif atas waktu dan biaya:

$$\min(s_{n+1}, \sum_{i \in I_0^C} z_{i,k}(x_{i,k}, \tau_{i,k}))$$

1. Cost-Driven (Deadline Terkunci)

Meminimalkan total biaya upah endogen dengan batas $T_{\max}$:

$$\min \sum_{i \in I_0^C} \sum_{k \in K} z_{i,k}(x_{i,k}, \tau_{i,k}) \quad \text{s.t.} \quad s_{n+1} \leq T_{\max}$$

2. Time-Driven (Anggaran Terkunci)

Meminimalkan durasi proyek dengan batas anggaran B :

$$\min s_{n+1} \quad \text{s.t.} \quad \sum_{i \in I} \sum_{k \in K} z_{i,k}(x_{i,k}, \tau_{i,k}) \leq B$$

Fungsi Objektif Bonus-Penalty

Dimensi “time” dikonversikan menjadi “cost” melalui mekanisme bonus dan penalty pada fungsi objektif *cost-driven*.

$$\min \underbrace{\sum_{i \in I_0^C} \sum_{k \in K} z_{i,k}(x_{i,k}, \tau_{i,k})}_{\text{Total Biaya Upah Endogen}} + \underbrace{c_{\text{late}} \max(0, s_{n+1} - T_{\max})}_{\text{Denda Keterlambatan}} - \underbrace{c_{\text{early}} \max(0, T_{\max} - s_{n+1})}_{\text{Bonus Penyelesaian Awal}}$$

Metode Penyelesaian

A. Pendekatan Multi-Objektif

Menggunakan base metode CP-SAT dengan wrapper metode ε -constraint. Ini berarti kita bisa membuat *pareto front* untuk masalah diskrit ini.

CP-SAT

Constraint Programming
– Satisfiability yang dibuat oleh Google untuk optimisasi *integer*.

ε -constraint

Konversi multi-objektif menjadi single-objektif dengan membatasi salah satu variabel keputusan.

B. Pendekatan Bonus–Penalty

Cukup menggunakan metode basis CP-SAT.

CP-SAT

Constraint Programming – Satisfiability yang dibuat oleh Google untuk optimisasi *integer*.

A. Pendekatan Multi-Objektif

Di sini, kita pikirkan dua jenis metode yang memungkinkan. Akan dicoba keduanya.

NSGA-II

Non-dominated Sorting Genetic Algorithm II berupa algoritma genetik untuk masalah multi-objektif. Ini dapat menghasilkan *pareto front* yang akurat, tetapi dengan biaya komputasi mahal.

Diskritisasi MILP + ε -constraint

Dilakukan diskritisasi nilai $x_{i,k}$ menjadi m nilai mungkin dan $\tau_{i,k}$ menjadi n nilai mungkin, sehingga diperoleh gridisasi $\xi_{i,k}^{m,n}$. Ini menjadi masalah **Multi-Modal RCPSP (MRCPSP)**.

B. Pendekatan Bonus-Penalty

Cukup menggunakan metode basis algoritma genetik.

Genetic Algorithm

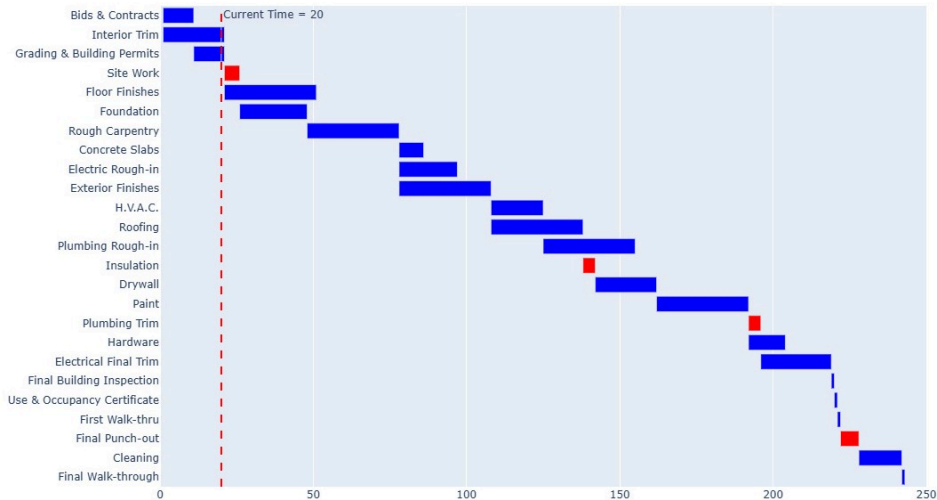
- **Representasi Kromosom:** Satuan gen adalah $(x_{i,k}, \tau_{i,k})$ sehingga kromosom adalah vektor semua pasangan.
- **Evaluasi Fitness:** Membangun jadwal dari kromosom, lalu menghitung fungsi objektifnya.
- **Operator Genetika:** Menggunakan operator seleksi turnamen, persilangan SBX (*Simulated Binary Crossover*), dan mutasi PM (*Polynomial Mutation*).

Hasil Eksplorasi

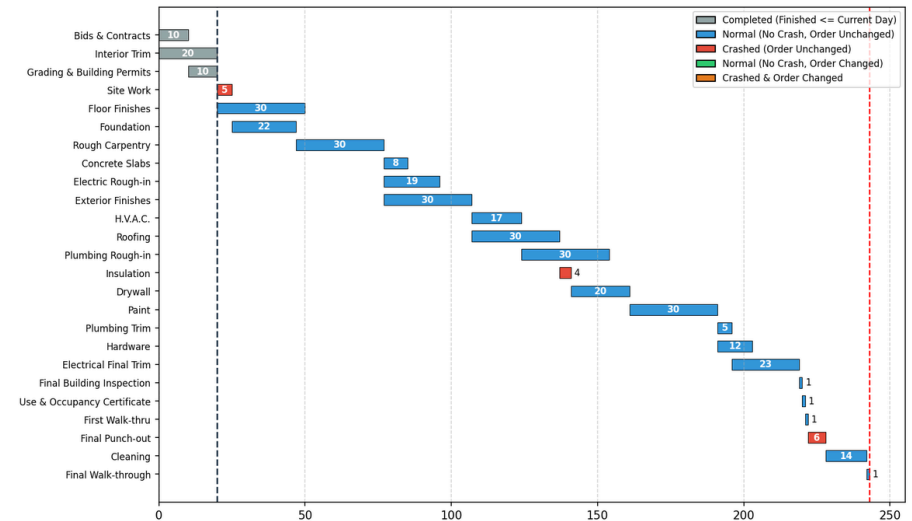
Hasil Eksplorasi Model 1

Digunakan **objective cost driven** dengan parameter $T_0 = 20$ dan $T_{\max} = 243$, diperoleh hasil berikut.

Gantt Chart Model IDSC



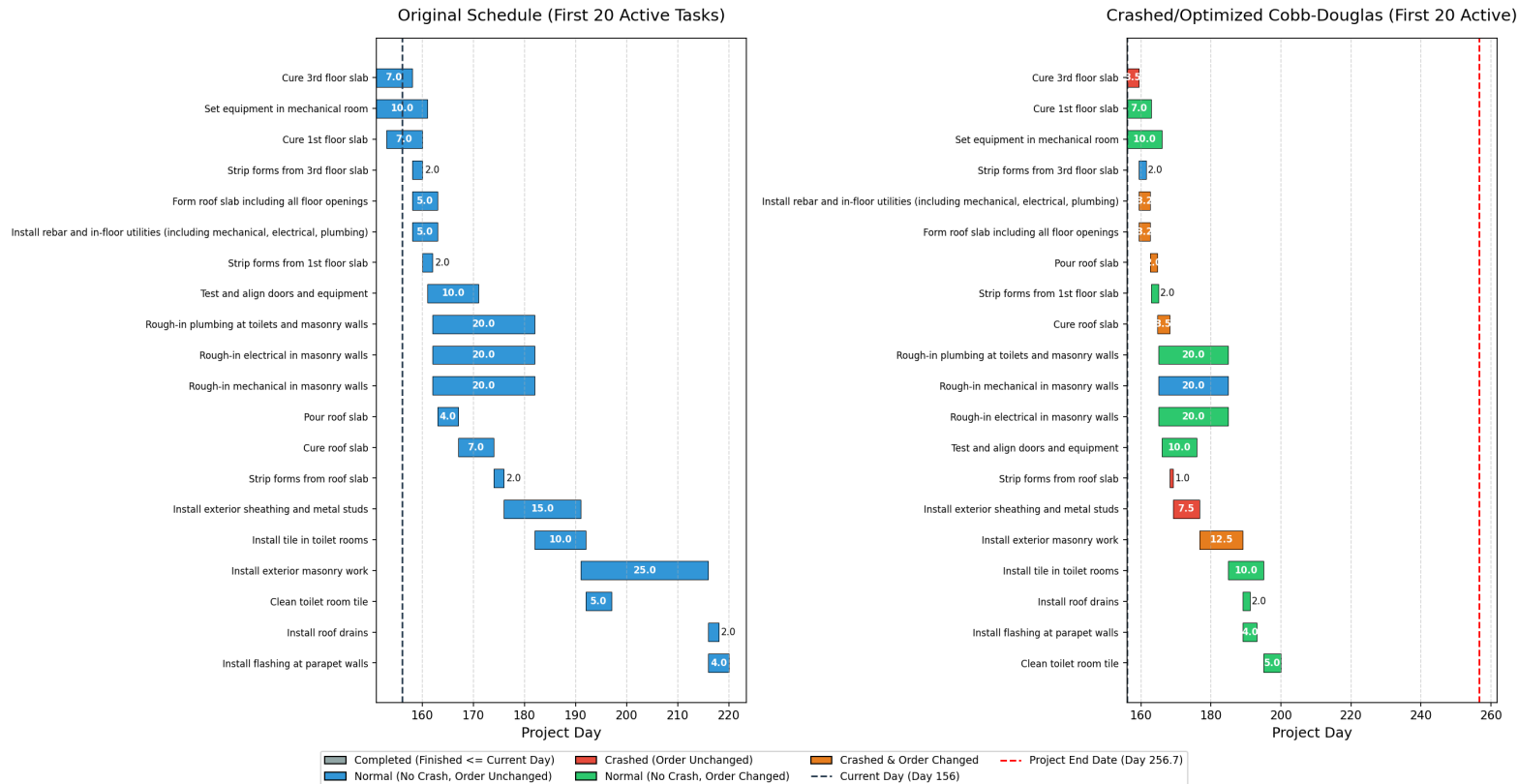
Gantt Chart Model Kami



Model	Total Biaya (\$)	Status Solusi	Waktu Eksekusi
IDSC	220	Optimal Global	~30 s
Kami	220	Optimal Global	< 1s

Hasil Eksplorasi Model 2

Digunakan **objective bonus-penalty** dengan parameter $T_0 = 156$ dan $T_{\max} = 344$, diperoleh hasil berikut (hanya ditampilkan 20 *task* aktif pertama untuk kejelasan) dengan waktu eksekusi ~30 menit.

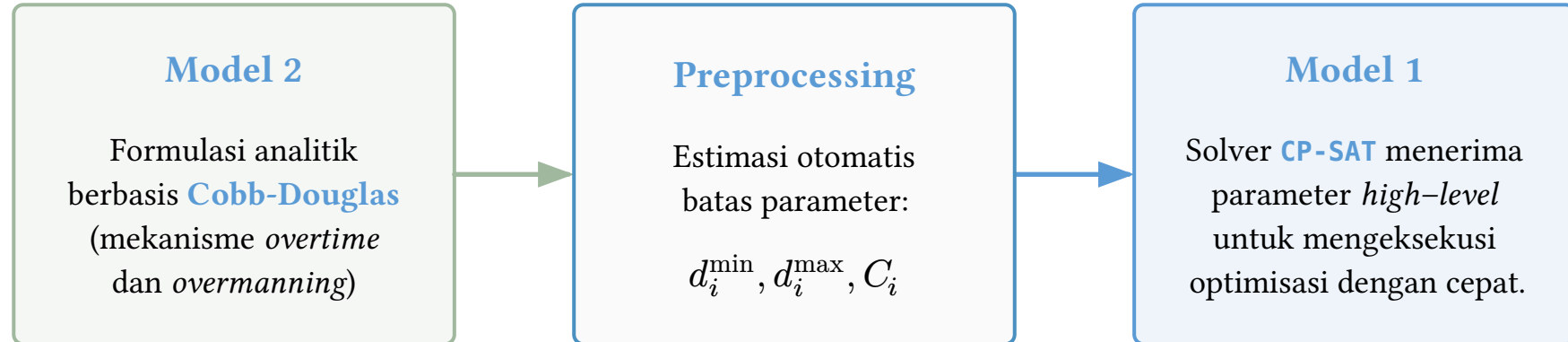


Biaya Baseline	\$506,210
Biaya Crashing	\$551,288

Skenario 3: Model Hybrid

Ide Utama:

Menggabungkan kelebihan **realisme data** Skenario 2 (mekanisme Cobb-Douglas) dengan **kecepatan komputasi** Skenario 1 (metode CP-SAT).



Langkah Preprocessing

Untuk setiap $i \in I, k \in K$ dihitung $(d_{i,k}^{(\max)}, d_{i,k}^{(\min)}, C_i)$ sebagai berikut.

Durasi Normal	$d_i^{\max} = \max_k \left\lceil \frac{W_{i,k}}{8U_{i,k}} \right\rceil$
Durasi Minimum	$d_i^{\min} = \max_{k \in K_i} \left\lceil \frac{W_{i,k}}{8U_{i,k}} \cdot \left(\frac{1}{x_{\max}} \right)^\alpha \cdot \left(\frac{8}{8 + \tau_{\max}} \right)^\beta \right\rceil$
Biaya <i>Crashing</i> Baseline	$Z_i^{(\text{base})} = \sum_k W_{i,k} r_k$
Biaya <i>Crashing</i> Maksimum	$Z_i^{(\text{crash})} = \sum_k z_{i,k}(x_{\max}, \tau_{\max})$
Biaya <i>Crashing</i> Harian	$C_i = \begin{cases} \frac{Z_i^{(\text{crash})} - Z_i^{(\text{base})}}{d_i^{\max} - d_i^{\min}}, & d_i^{(\max)} > d_i^{(\min)}, \\ 0, & d_i^{(\max)} = d_i^{(\min)}. \end{cases}$

Setelah preprocessing, kita kembali selesaikan Model 1 tetapi dengan nilai $d_{i,k}^{(\max)}$, $d_{i,k}^{(\min)}$, C_i yang baru.

Project Objective

$$\min \sum_{i \in I_0^C} C_i \left(d_i^{(\max)} - (e_i - s_i) \right) + c_{\text{late}} \max(0, s_{n+1} - T_{\max}) - c_{\text{early}} \max(0, T_{\max} - s_{n+1})$$

Temporal Constraints

$$s_j \geq e_i$$

$$d_i^{\min} \leq e_i - s_i \leq d_i^{\max}$$

Resource Constraints

$$\sum_i U_{i,k} \cdot \mathbb{1}\{s_i \leq s_j < e_i\} \leq U_k^{\max}$$

Karakteristik	Model Baseline (Skenario 1)	Model Baru (Skenario 2)	Model Hybrid (Skenario 3)
Input Biaya	Eksplisit per hari crashing (\$/hari).	Implicit dari upah SDM (\$/jam).	Cobb-Douglas preprocessing + linearisasi crash slope.
Percepatan	Memotong hari durasi secara langsung.	Menambah orang (x) & jam kerja lembur (τ).	Memotong hari durasi secara langsung.
Sifat Efisiensi	Efisiensi konstan (biaya linier terhadap crashing).	Efisiensi menurun (diminishing returns).	Efisiensi Cobb-Douglas didekati secara lokal.
Tipe Precedence	<i>Finish-to-Start</i> (FS)	<i>Finish-to-Start</i> (FS).	<i>Finish-to-Start</i> (FS).
Metode Solver	CP-SAT (OR-Tools)	MILP (Pyomo) vs GA (Pymoo)	CP-SAT (OR-Tools) setelah preprocessing (Python)

Kesimpulan

1. Skenario 1 (Diskret Linear)

- Berkinerja sangat cepat dengan metode CP-SAT
- Data biaya crashing per hari jarang dimiliki perusahaan

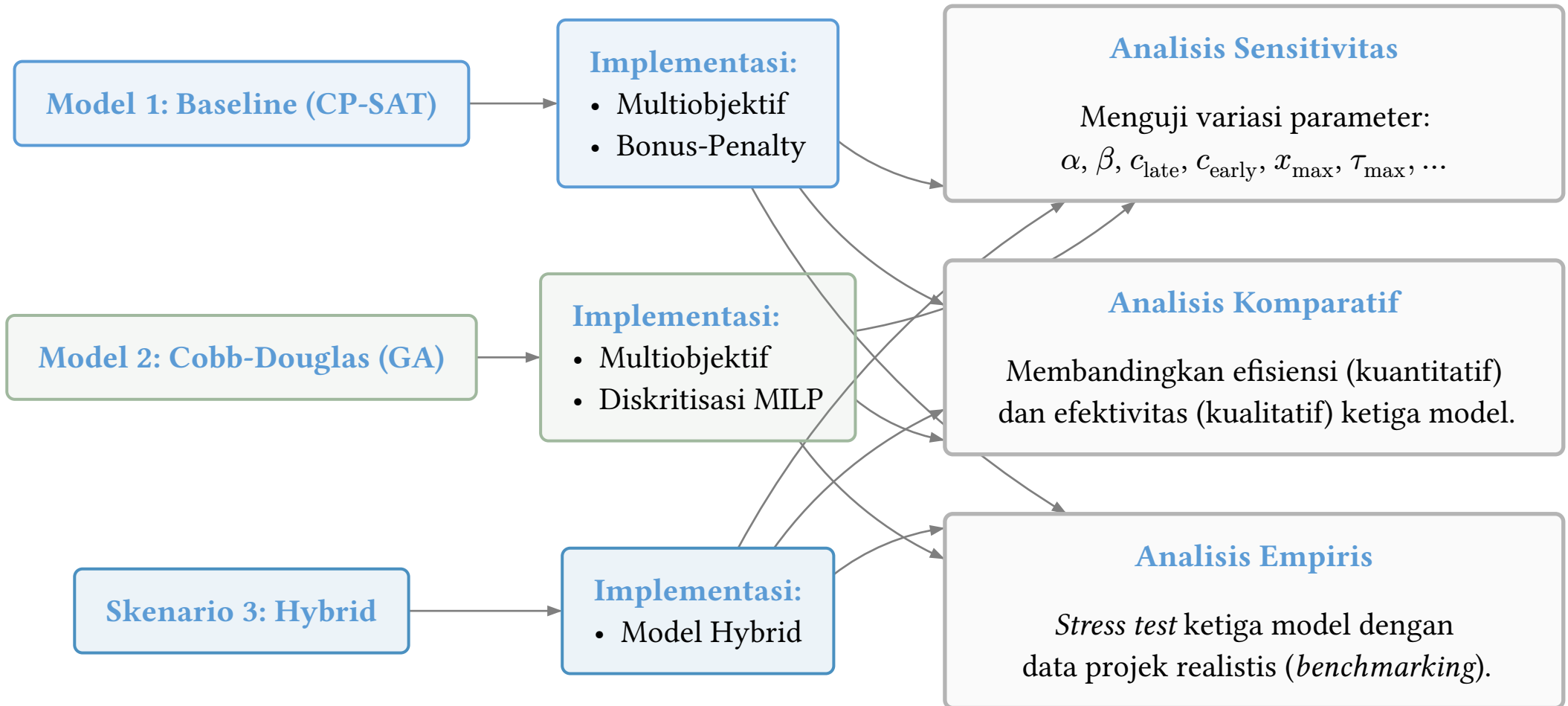
2. Skenario 2 (Cobb-Douglas):

- Memodelkan realitas penambahan tenaga kerja dan lembur
- Menggunakan mekanisme Cobb-Douglas
- Biaya komputasi yang tinggi dengan metode GA

3. Skenario 3 (Hybrid):

- Menggunakan preprocessing Cobb-Douglas dari Skenario 2
- Menghasilkan parameter biaya dan durasi berdasarkan data yang sudah ada tanpa data *crash cost* yang lengkap
- Dapat diselesaikan dengan cepat menggunakan CP-SAT.

Rencana Selanjutnya



Terima Kasih
